AI OPERATIONS (AIOps) · MODULE 4

Serving on Kubernetes with Minikube

Module 4 · Deck 4 of 4
Deployments · Services · probes · rolling update & rollback · canary & blue-green · HPA — 3 nodes, CPU only

Third-year Data Science · CPU-only · all code runs on a laptop

Learning outcomes for this deck

Build a container image that bakes in the model and run it on a 3-node minikube cluster

Write Deployment, Service and HPA manifests and explain every field that matters

Explain readiness vs liveness probes and how they protect users during failures and rollouts

Release with rolling update, rollback, replica-ratio canary and blue-green

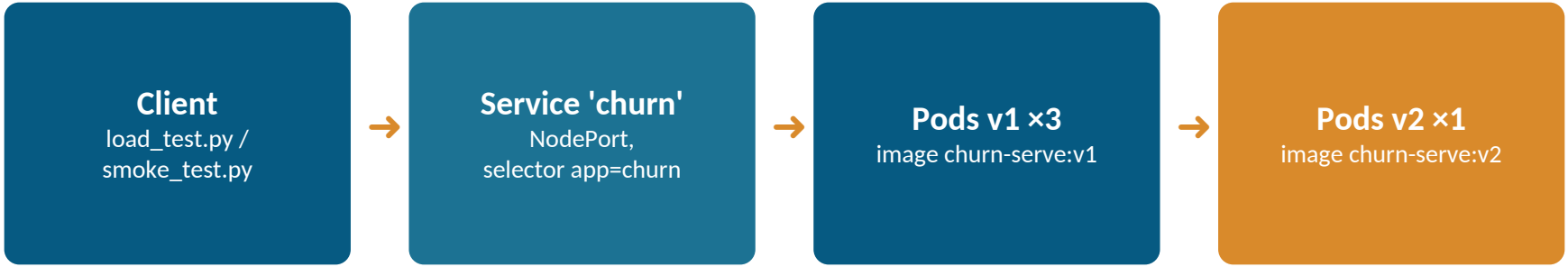
Autoscale with the HPA and predict its behaviour from the formula

Automate a post-deploy smoke test that triggers rollback (CI/CD sanity check)

Module 3 recap → Module 4 use

Kubernetes object	You used it for (Module 3)	We use it today for
Pod	One container running a training worker	One replica of the churn API
Job	Parallel hyper-parameter search	— (not for online serving)
Deployment	Long-running predictor, replicas	v1 and v2 ReplicaSets, rolling update, rollback
Service	Stable address + load balancing	Canary by replica ratio; blue-green selector switch
HPA	(introduced)	Autoscale on CPU utilization
Probes	(introduced)	Gate traffic (readiness) and restart (liveness)

What we are building



Three minikube nodes (node containers on your laptop). Pods from both Deployments carry the label app=churn, so one Service balances across all four: ≈ 75 % v1 / 25 % v2. A second Service (churn-bg) selects ONE version for blue-green. An HPA scales the v1 Deployment between 2 and 6 pods.

1

Cluster and images

3 nodes · images built inside the cluster

Prerequisites and budget

Item	Requirement	Check
Docker	Engine/Desktop running	docker info
minikube + kubectl	Recent versions	minikube version; kubectl version --client
RAM / CPU	≈ 6 GB and 6 vCPUs free for 3 nodes (2 CPUs, 2.2 GB each)	Docker Desktop → Resources
Smaller laptop?	Use --nodes 2, or run notebook 4d sections 1–4 only	—
Python env	train_models.py must have produced data/model_v*.joblib	ls data/

If Docker Desktop is capped below 6 GB, nodes may fail to start (Exit 137 / OOM). Raise the cap before starting the cluster.

Create the cluster

```bash
minikube start -p aiops --nodes 3 --cpus 2 --memory 2200 --driver=docker
minikube -p aiops addons enable metrics-server        # needed by the HPA and `kubectl top`

kubectl get nodes -o wide
# NAME          STATUS  ROLES          ...
# aiops         Ready   control-plane
# aiops-m02     Ready   <none>
# aiops-m03     Ready   <none>

kubectl config current-context                        # -> aiops
# tear down at the end of the day (frees ~6 GB):
minikube delete -p aiops
```

Untested in the authoring sandbox. Start-up takes 2–4 minutes the first time (image pulls).

Dockerfile.serve — model baked into the image

Dockerfile

```
FROM python:3.11 AS builder
WORKDIR /build
COPY requirements-serve.txt .
RUN pip install --no-cache-dir --target=/build/deps -r requirements-serve.txt

FROM python:3.11-slim
ARG MODEL_VERSION=v1
WORKDIR /app
COPY --from=builder /build/deps /usr/local/lib/python3.11/site-packages
ENV PATH="/usr/local/lib/python3.11/site-packages/bin:${PATH}" PYTHONPATH=/usr/local/lib/python3.11/site-packages
COPY rest_app.py .
COPY data/model_${MODEL_VERSION}.joblib ./model.joblib
ENV MODEL_PATH=/app/model.joblib
EXPOSE 8000
CMD ["python", "-m", "uvicorn", "rest_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Multi-stage build (Module 3). The model travels INSIDE the image: no shared storage, identical on every node.

Build the images inside the cluster

```bash
python train_models.py                    # creates data/model_v1.joblib, model_v2.joblib

minikube -p aiops image build -t churn-serve:v1 \
      --build-opt=build-arg=MODEL_VERSION=v1 -f Dockerfile.serve .
minikube -p aiops image build -t churn-serve:v2 \
      --build-opt=build-arg=MODEL_VERSION=v2 -f Dockerfile.serve .

minikube -p aiops image ls | grep churn-serve

# if a pod later shows ErrImageNeverPull on one node:
minikube -p aiops image load churn-serve:v1
```

No registry needed. That is why the manifests say imagePullPolicy: Never. (Alternative: docker build locally, then `minikube image load`.)

2

Manifests

Every field you need to understand

deployment-v1.yaml (1/2) — strategy and placement

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: churn-v1, labels: {app: churn, version: v1}}
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  selector: {matchLabels: {app: churn, version: v1}}
  template:
    metadata: {labels: {app: churn, version: v1}}
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: ScheduleAnyway
        labelSelector: {matchLabels: {app: churn, version: v1}}
```

maxUnavailable: 0 → capacity never dips during a rollout. topologySpreadConstraints spreads pods across nodes (ScheduleAnyway = best effort).

deployment-v1.yaml (2/2) — container, resources and probes

```yaml
containers:
- name: server
  image: churn-serve:v1
  imagePullPolicy: Never
  ports: [{containerPort: 8000}]
  env:
  - {name: POD_NAME, valueFrom: {fieldRef: {fieldPath: metadata.name}}}
  resources:
    requests: {cpu: 100m, memory: 128Mi}
    limits: {cpu: 500m, memory: 384Mi}
  readinessProbe: {httpGet: {path: /healthz, port: 8000}, initialDelaySeconds: 3, periodSeconds: 5}
  livenessProbe: {httpGet: {path: /healthz, port: 8000}, initialDelaySeconds: 10, periodSeconds: 10}
```

Downward API gives each pod its own name (shows up as served_by). requests drive scheduling and HPA maths; limits throttle.

Readiness vs liveness — two different questions

Readiness: 'may I send you traffic?'

Fails → pod REMOVED from the Service endpoints. Not restarted.
Use for: still loading the model, temporarily overloaded.
Ours: GET /healthz every 5 s.

Liveness: 'are you stuck?'

Fails (repeatedly) → container RESTARTED.
Use for: deadlock, wedged process.
Ours: GET /healthz every 10 s, after 10 s.

Anti-patterns

- Liveness that depends on a downstream DB → cascading restarts
- Same aggressive probe for both
- Probe path your app doesn't serve → CrashLoopBackOff
- async def + CPU-bound predict → probe times out under load

service.yaml and the v2 canary Deployment

```yaml
# One Service for ALL churn pods (selector has NO version label).
# => traffic splits roughly in proportion to replica counts: v1 x3 + v2 x1 ~ 75% / 25%  (canary by replicas)
apiVersion: v1
kind: Service
metadata: {name: churn, labels: {app: churn}}
spec:
  type: NodePort
  selector: {app: churn}
  ports: [{port: 80, targetPort: 8000, name: http}]

# deployment-v2.yaml = deployment-v1.yaml with:
#   name churn-v2 · label version: v2 · image churn-serve:v2 · replicas: 1
# Both sets of pods carry app=churn -> the Service balances over all 4 pods.
```

Canary by replica ratio: 3 v1 + 1 v2 ⇒ ≈ 25 % of requests reach v2. Coarse (minimum share = 1/N), but needs no extra infrastructure.

Apply and verify

```bash
kubectl apply -f k8s/deployment-v1.yaml -f k8s/deployment-v2.yaml -f k8s/service.yaml
kubectl rollout status deploy/churn-v1 --timeout=180s
kubectl rollout status deploy/churn-v2 --timeout=180s

kubectl get pods -l app=churn -o wide       # NODE column: spread over 3 nodes?
kubectl get endpoints churn                 # 4 pod IPs behind the Service
kubectl describe pod -l version=v2 | grep -A5 Events

# reach the NodePort from your laptop (docker driver) — keep this terminal open:
minikube -p aiops service churn --url       # e.g. http://127.0.0.1:53412

URL=http://127.0.0.1:53412
curl -s $URL/healthz; curl -s $URL/v1/models/churn
```

Pods stuck in Pending? → `kubectl describe pod`: usually insufficient CPU/memory or an unschedulable constraint.

Load test the Service and check the split

```bash
python load_test.py --url $URL/v1/models/churn:predict --concurrency 1 4 8 --duration 5

#  concurrency  requests  rps  p50_ms  p95_ms  p99_ms  errors  replicas  versions
#       ...                                                 4     {'v1': ~75%, 'v2': ~25%}
#                                                  ^ distinct pods that answered

# what you should see (expected, not measured by the authors on a real cluster):
#  • replicas = 4 (all pods serve), versions ≈ 3:1
#  • throughput higher than one pod; latency includes the NodePort / docker-network hop
#  • the same percentiles-vs-concurrency knee as Deck 1, shifted right by the pod count

# local simulation of exactly this logic (notebook 4d §3, measured):  v1 74 %  v2 26 %
```

Honesty note for the instructor: the lines marked 'expected' are predictions from the manifests, not cluster output we observed.

3

Operating the service

Scale · self-heal · roll · roll back · switch

Scale and self-heal

```bash
kubectl scale deploy/churn-v1 --replicas=6
kubectl get pods -l app=churn,version=v1 -o wide     # spread across 3 nodes by the constraint

# self-healing: delete a pod, the ReplicaSet recreates it
kubectl delete pod -l app=churn,version=v2 --wait=false
kubectl get pods -l version=v2 -w                    # Terminating -> ContainerCreating -> Running 1/1

# node failure (instructor demo): cordon + drain a worker, watch pods reschedule
kubectl drain aiops-m03 --ignore-daemonsets --delete-emptydir-data
kubectl uncordon aiops-m03
```

Deployment controller: desired replicas = actual replicas, forever. Readiness keeps the new pod out of rotation until it can answer.

Rolling update and rollback

```bash
kubectl set image deploy/churn-v1 server=churn-serve:v2        # new template -> new ReplicaSet
kubectl rollout status deploy/churn-v1 --timeout=180s
kubectl rollout history deploy/churn-v1

kubectl rollout undo deploy/churn-v1                          # ROLLBACK to the previous ReplicaSet
kubectl rollout status deploy/churn-v1

# safety net: a BROKEN image never becomes Ready, so the rollout stalls and old pods keep serving
kubectl set image deploy/churn-v1 server=churn-serve:does-not-exist
kubectl rollout status deploy/churn-v1 --timeout=30s          # times out; pods ErrImageNeverPull
kubectl rollout undo deploy/churn-v1
```

maxSurge: 1 + maxUnavailable: 0: start one new pod → wait for readiness → remove one old pod → repeat. Users never see reduced capacity.

Blue-green: switch the Service selector

yaml / bash

```
apiVersion: v1
kind: Service
metadata: {name: churn-bg, labels: {app: churn}}
spec:
  type: NodePort
  selector: {app: churn, version: v1}
  ports: [{port: 80, targetPort: 8000, name: http}]

kubectl apply -f k8s/service-bluegreen.yaml
kubectl patch svc churn-bg -p '{"spec":{"selector":{"app":"churn","version":"v2"}}}'  # green
kubectl patch svc churn-bg -p '{"spec":{"selector":{"app":"churn","version":"v1"}}}'  # rollback
kubectl get endpoints churn-bg
```

Both stacks run side by side (double the capacity cost); the switch is atomic and reversible. Always smoke-test green BEFORE switching.

Release strategies compared

Strategy	Traffic to new version	Rollback speed	Extra cost	Granularity
Rolling update	Gradual, replaces old pods	Minutes (undo rollout)	+1 surge pod	Pods
Canary (replica ratio)	≈ new/(new+old) pods	Seconds (scale v2 to 0)	Small	Coarse: min 1/N
Blue-green	0 % → 100 % at once	Instant (selector back)	2× capacity	All-or-nothing
Shadow	Copy only; answers ignored	n/a (no user impact)	Extra compute	Per request
Weighted canary (mesh/Ray router)	Exact %, e.g. 1 %	Seconds	Needs mesh / router	Per request

4

Autoscaling

Horizontal Pod Autoscaler

hpa.yaml and the formula

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: churn-v1}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: churn-v1}
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource: {name: cpu, target: {type: Utilization, averageUtilization: 50}}
  behavior:
    scaleDown: {stabilizationWindowSeconds: 60}

# desiredReplicas = ceil( currentReplicas × currentUtilization / targetUtilization )
# utilization = CPU usage / CPU REQUEST (100m) — it can exceed 100 %
# sync period 15 s · 10 % tolerance · scale up fast · scale down to the highest recommendation of the last 60 s
```

Example: 2 pods at 100 % of request, target 50 % → ceil(2 × 100/50) = 4 pods.

HPA behaviour simulated in Python (notebook 4d §4)

Tick (15 s)	Traffic (rps)	Utilization of request	Replicas
0	20	25 %	2
6	60	75 %	3
10	140	58 %	6 (cap)
18	300	125 %	6 (cap)
28	60	25 %	3 (after window)

Simulation parameters: 40 rps per pod at 100 % of request, target 50 %, min 2, max 6, 60 s scale-down window. Lessons: the count LAGS the ramp; it caps at maxReplicas while utilization stays above target (plan for peak: 8 pods would hold 300 rps below 100 %); it holds capacity through the stabilization window.

Drive the HPA with load (real cluster)

```bash
kubectl apply -f k8s/hpa.yaml
kubectl get hpa churn-v1                        # TARGETS shows <unknown> for ~1 min: normal
python load_test.py --url $URL/v1/models/churn:predict --concurrency 16 --duration 120

# in a second terminal
kubectl get hpa churn-v1 -w                     # watch TARGETS and REPLICAS change
kubectl top pods -l app=churn                   # needs metrics-server
kubectl describe hpa churn-v1 | tail -15        # events: SuccessfulRescale ...

# after the load stops, replicas fall back only after the 60 s stabilization window
```

TARGETS stuck at <unknown>? metrics-server isn't ready (`kubectl -n kube-system get pods`) or the pod has no CPU request.

5

Release safety nets

Smoke tests, simulation and what we could not test

CI/CD sanity check → automatic rollback

```bash
# smoke_test.py (tested locally): healthz, expected version, 30 predictions, p95 bound, bad input -> 422
python smoke_test.py --url $URL --expect-version v2 --max-p95-ms 100
# SMOKE TEST PASSED (version=v2, p95=13.8 ms)          exit 0
# SMOKE TEST FAILED: version v1 != expected v2; ...     exit 1

# pipeline step (shell):
kubectl set image deploy/churn-v1 server=churn-serve:v2
kubectl rollout status deploy/churn-v1 --timeout=180s || { kubectl rollout undo deploy/churn-v1; exit 1; }
python smoke_test.py --url $URL --expect-version v2 --max-p95-ms 100 \
  || { echo "smoke test failed -> rollback"; kubectl rollout undo deploy/churn-v1; exit 1; }
```

Two gates: Kubernetes' readiness (does it start?) and OUR smoke test (is it correct and fast?). Either failing triggers rollback — syllabus: CI/CD sanity checks.

What we simulated locally vs what needs a cluster

Tested here (notebook 4d §1–4, 6)

- Manifest lint: labels, selectors, probe paths, CPU fit
- Local 'pods' + endpoint controller: 74 %/26 % split, kill → removed in ~0.5 s, restarted
- HPA algorithm simulation
- Canary planner, HPA sizing

Needs minikube (§5)

- Image build into the cluster
- kubectl apply / rollout / undo
- NodePort access
- HPA with metrics-server
- 3-node spreading

Printed-only when no cluster is found

Report back

The authors could not run §5 (no Docker daemon). If any command or flag needs fixing on your OS, tell the instructor — treat the slides as a runbook to be validated.

Common errors and fixes

Symptom	Likely cause	Fix
ErrImageNeverPull	Image not present on that node	minikube -p aiops image load churn-serve:v1; check the image name/tag
Pod Pending	Not enough CPU/memory or constraint unsatisfiable	kubectl describe pod → Events; lower requests or add resources
CrashLoopBackOff	App crashes / liveness path wrong / model file missing	kubectl logs <pod> --previous
Service URL not reachable	Docker driver needs a tunnel	Keep `minikube service churn --url` running, or use `kubectl port-forward svc/churn 8080:80`
HPA TARGETS <unknown>	metrics-server not ready / no CPU request	Wait; check kube-system pods; set resources.requests.cpu
Rollout stuck	New pods never Ready	kubectl rollout status; kubectl describe; kubectl rollout undo
Cluster won't start (Exit 137)	Docker memory cap too low	Raise Docker Desktop memory; use --nodes 2

Module 4 in one table — four ways to serve the same model

	FastAPI	MLflow serve	Ray Serve	Kubernetes
Who writes the API?	You	MLflow (from signature)	You (Python)	Your container
Versioning	Return model_version	Registry aliases	Per-deployment	Image tags, ReplicaSets
Scale-out	--workers	Manual	replicas + autoscaling	replicas + HPA, multi-node
Canary	—	Shadow (2 servers)	Router weight (exact %)	Replica ratio / mesh
Rollback	Redeploy	Alias + restart	Redeploy router	rollout undo / selector
Self-healing	Process manager	—	Controller	ReplicaSet + probes
Needs containers?	No	Optional	No	Yes

HANDS-ON

Notebook 4d — Minikube serving (core ≈ 20 min)

1. §1: toolchain check — does your machine have docker, minikube, kubectl (and a running daemon)?
2. §2: manifest lint — all checks green? Break a label on purpose and re-run to see it caught
3. §3: local 'pods' + endpoint controller: observe the 75/25 split, kill a pod, read the event log
4. §4: HPA simulation — find the maxReplicas that holds 300 rps; then change the scale-down window
5. §5 (if you have a cluster): run 5.1–5.4, and capture `kubectl get pods -o wide` and the load-test table
6. Take-home: YOUR TURN A (canary planner) and B (size the HPA)

Notebook: Lecture4d_Minikube_Serving_Exercise.ipynb

Recap — Kubernetes serving, and Module 4

- Deployment = desired replicas + rolling update + rollback history; Service = stable address across READY pods
- Readiness gates traffic; liveness restarts; both must hit an endpoint the app really serves
- Canary by replica ratio is coarse; blue-green by selector patch is instant and reversible
- HPA = ceil(current × util/target) with tolerance, lag and stabilization — size maxReplicas for peak
- Gate every release with a smoke test and automatic rollback

NEXT → Assignment + 7-question quiz (Google Form)